TOWARDS A UNIVERSAL FEATURE EXTRACTOR FOR ATARI GAMES

Reconstruction, FSQ and Joint-Embedding Prediction

Sajid Ali Sadruddin

Technische Universität Dresden (TUD)

September, 2026



1. Introduction
2. Data & Games
3. Models
4. Methodology
5. Results
6. Conclusion

1 Introduction

2 Data & Games

3 Models

4 Methodology

5 Results

6 Conclusion

Introduction

- Atari games are one of the most common benchmarks for RL from pixels
- Every game gives the agent the same input: 4 stacked 84×84 grey-scale frames
- Still, each agent trains its own CNN feature extractor (NatureCNN) from scratch, using only the game reward
- Many games look alike: sprites, balls, walls and scrolling backgrounds
- **Idea:** pre-train **one** encoder on many games and reuse it for PPO on every game

Motivation and Research Questions

- A shared extractor could save training steps and compute, work on new games, and give one latent space for all games
- I looked at two design choices:
  - **Objective:** reconstruct the pixels (VAE, VQ-VAE, FSQ-VAE) or predict the next state in latent space (V-JEPA)
  - **Usage:** keep the encoder *frozen* or *fine-tune* it together with the policy

Research questions

RQ1 Which objective gives the most useful features for PPO?
RQ2 Is it better to freeze or to fine-tune the encoder?
RQ3 Do the features work on games not seen in pre-training?
RQ4 What does it cost, and can we interpret the features?

1 Introduction

2 Data & Games

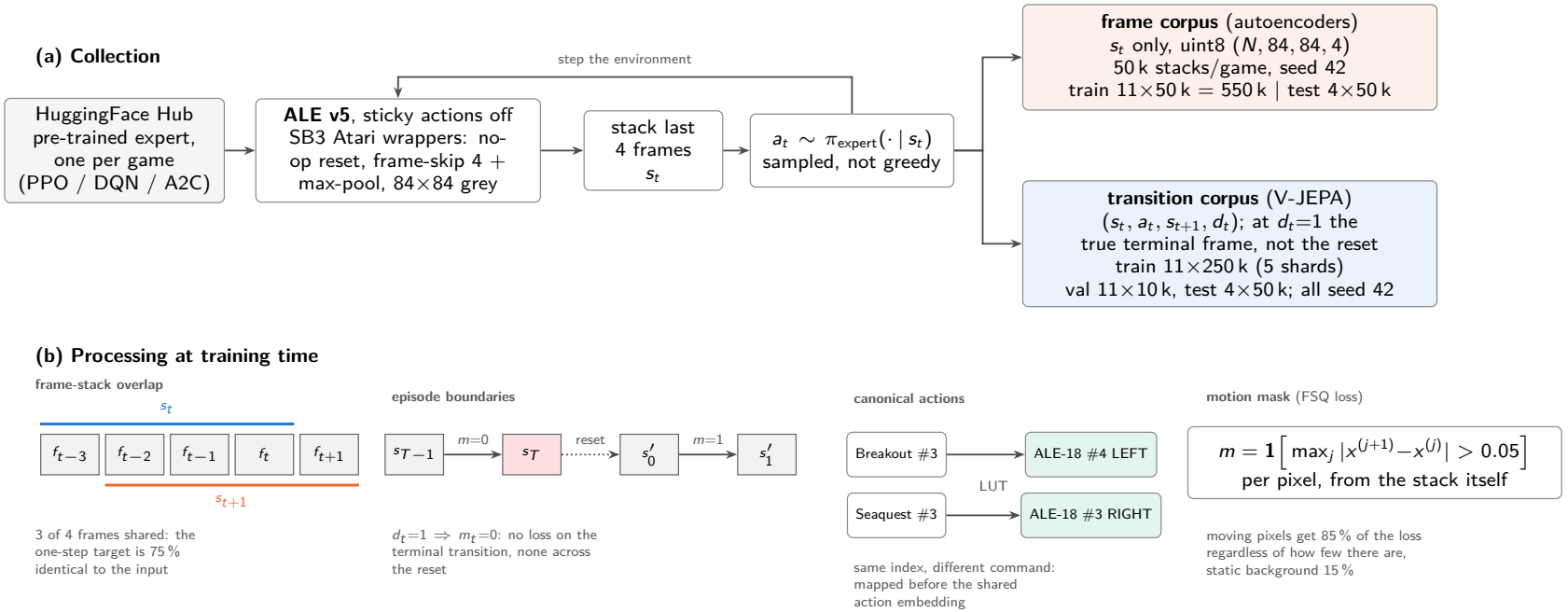
3 Models

4 Methodology

5 Results

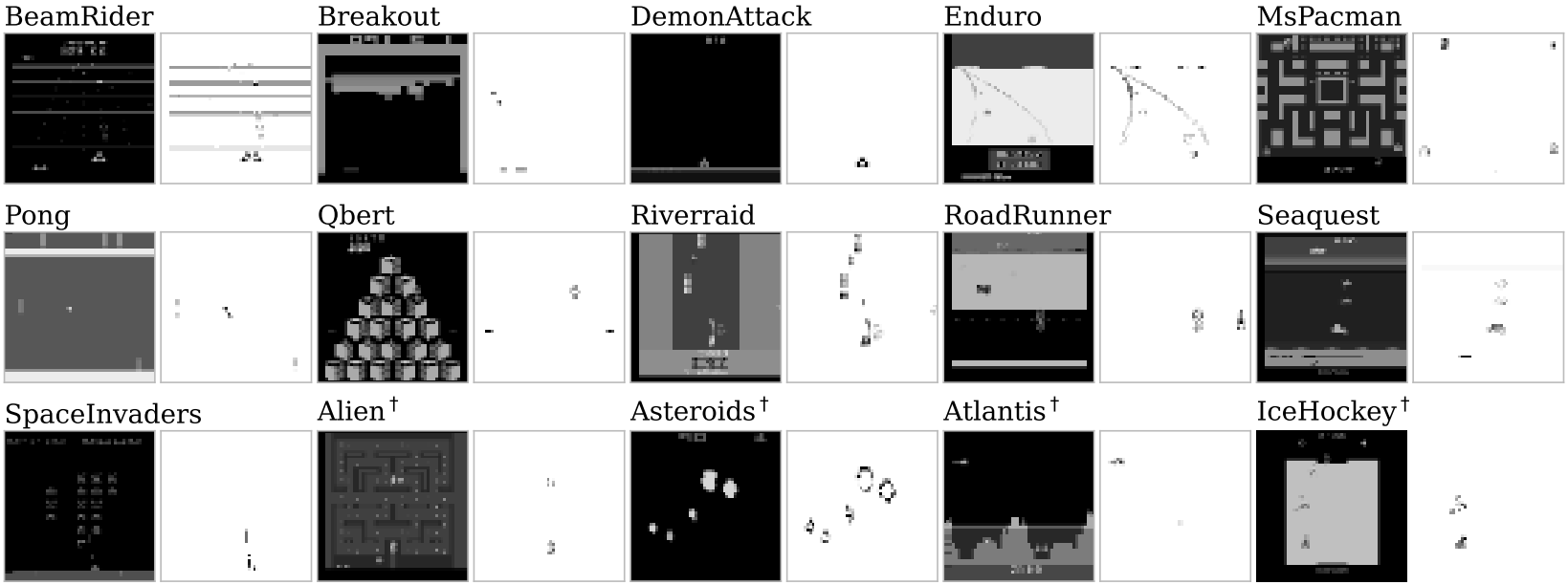
6 Conclusion

Data Collection



- Random play rarely leaves the first screen, so I used expert agents from HuggingFace (actions sampled, so some play is sub-optimal)
- **Frame dataset:** 50 k frame stacks per game, for the autoencoders
- **Transition dataset:** 2.75 M (s_t, a_t, s_{t+1}, d_t) from 11 games, for the V-JEPA
- 11 games for pre-training, 4 games held out for testing

The Games



Frame t (left) and the change to the next frame (right) for all 15 games; † held-out games

How Different the Games Are

- **Appearance:** mean brightness goes from 5 (DemonAttack) to 105 (Pong), and 1% to 95% of the pixels are black
- **Motion:** in most games less than 4% of the pixels move (Pong 1.7%, Breakout 1.4%)
  - The objects that matter for playing are a few small sprites
- **Actions:** 4 to 18 actions per game, and the same index means a different button in different games
  - I mapped all games to one common set of 18 ALE actions
- **Episodes:** from about 120 steps (Atlantis) to 12 500 steps (Enduro)
- **Expert data:** the favourite action makes up 67% of the steps in RoadRunner and 99% in Asteroids, so there are few examples of other actions

1 Introduction

2 Data & Games

3 Models

4 Methodology

5 Results

6 Conclusion

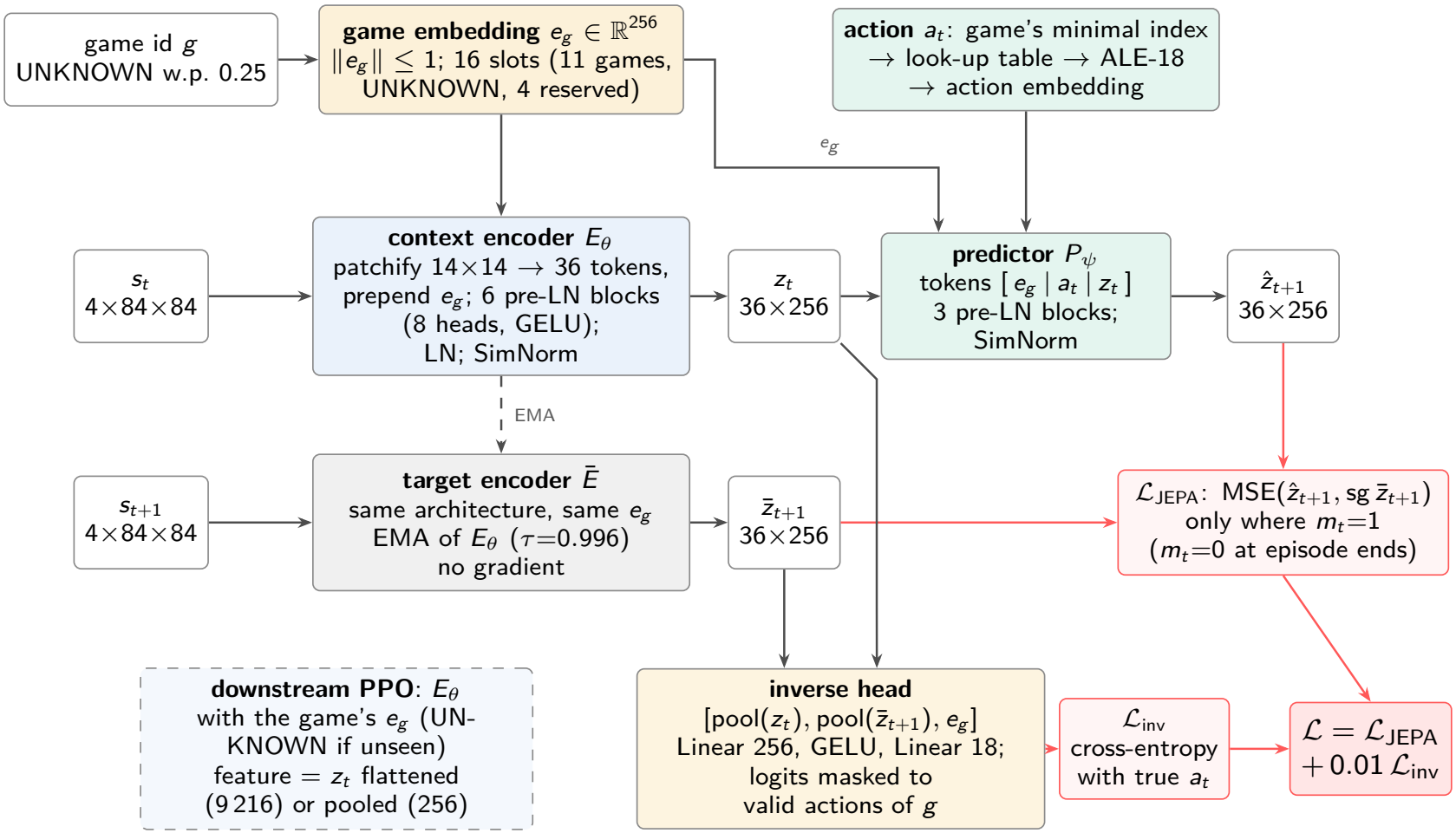
Autoencoders

- **VAE:** NatureCNN encoder with a 256-dim latent vector, but its 7×7 feature map is too coarse for small sprites (one cell $= 12 \times 12$ pixels)
- **Residual VQ-VAE:** finer 21×21 grid, each cell replaced by one of $K = 512$ codebook vectors
  - Small moving objects went missing, so I trained a version where moving pixels count $10\times$
- **FSQ-VAE:** each of 4 channels is simply rounded to $L = [8, 5, 5, 3]$ levels
  - No codebook to learn, so no unused codes
  - New decoder (upsampling + conv) to remove checkerboard patterns
  - Loss that gives moving pixels 85% of the weight:

$$\mathcal{L}_{\text{bal}} = 0.85 \, \frac{\sum m \odot (\hat{x} - x)^2}{\sum m} + 0.15 \, \frac{\sum (1-m) \odot (\hat{x} - x)^2}{\sum (1-m)}$$

- Feature for PPO: the flattened $4 \times 21 \times 21$ grid (1 764 values)

V-JEPA: Architecture



The model predicts the *latent* of the next frame stack, not its pixels. The target encoder is a slowly updated copy of the context encoder (no gradients). Only the context encoder is used for PPO.

V-JEPA: What I Changed (v1 → v2)

- v1 looked fine by its loss curve, but when I checked the pipeline I found bugs (inconsistent action indices, wrong axis order, a one-game validation set)
- **v2**, based on TD-MPC2:
  - one common set of 18 actions
  - a learnable embedding per game ("UNKNOWN" 25% of the time)
  - SimNorm: latents in groups of 8 with a softmax, against collapse
  - a head that guesses the action: $\mathcal{L} = \mathcal{L}_{\text{JEPA}} + 0.01\,\mathcal{L}_{\text{inv}}$

Part	Params	learns from $\mathcal{L}_{\text{JEPA}}$	from $\mathcal{L}_{\text{inv}}$
Context encoder	4.95 M	yes	yes
Predictor	2.37 M	yes	no
Inverse head	0.20 M	no	yes
Target encoder	4.95 M	moving average only	

1 Introduction

2 Data & Games

3 Models

4 Methodology

5 Results

6 Conclusion

Methodology: Pre-Training

	Std. VAE	Res. VQ-VAE	FSQ-VAE	V-JEPA v2
Data	550 k stacks	550 k stacks	550 k stacks	2.75 M transitions
Length	58 epochs	29 epochs	125 epochs	92 epochs
Steps	250 k	250 k	537 k	494 k
Batch	128	64	128	512
Optimiser, LR	Adam, 10^{-3}	$3{\cdot}10^{-4}$	AdamW, $3{\cdot}10^{-4}$	AdamW, $3{\cdot}10^{-4}$
Parameters	2.56 M	0.70 M	0.84 M	12.5 M
Hardware, time	H100, 0.8 h	H100, 0.9 h	H100, 1.9 h	H100, 15.6 h

- The architectures follow the original papers (VQ-VAE, FSQ, I-JEPA, TD-MPC2)
- The JEPA papers train huge models on many GPUs, so I used a smaller batch and a lower learning rate for my 12.5 M model on one GPU
- For the V-JEPA I used the checkpoint with the best validation loss after the early, near-collapsed phase (epoch 90), and a small sweep confirmed the inverse-loss weight $\lambda = 0.01$

Methodology: PPO and Evaluation

- **Baseline:** NatureCNN trained from scratch
- **Frozen:** pre-trained encoder stays fixed, only the actor and critic learn
- **Fine-tuned:** pre-trained encoder keeps learning together with the policy
- **PPO:** standard RL-Zoo Atari settings, 10 M steps, same 512-unit heads for every encoder
- **Metric:** mean return over the last 1 M steps, turned into the human-normalised score so that games can be compared

$$\text{HNS} = \frac{\text{score} - \text{random}}{\text{human} - \text{random}}$$

- One seed per run; all numbers come from the full Weights & Biases logs, except the Std-VAE and Res-VQ scores and the NatureCNN and V-JEPA held-out scores (recorded rounded)

1 Introduction

2 Data & Games

3 Models

4 Methodology

5 Results

6 Conclusion

Results: Pre-Training

- Error on held-out games was lowest **early**: the FSQ-VAE was best after 3 epochs and ended 21% worse after 125
- The V-JEPA's held-out loss ended 3× higher than on seen games, and its held-out action accuracy fell below chance, so both models fit the training games
- The V-JEPA validation loss **went up** after the first epoch while the effective rank (how spread out the latents are) grew from 1.4, below an untrained model's 2.5, to 57, and the action accuracy from 23% to 58% (chance 13%)
  - The low early loss came from almost collapsed latents
- Reconstruction: the FSQ-VAE had 3× less error on moving pixels than the VQ-VAE

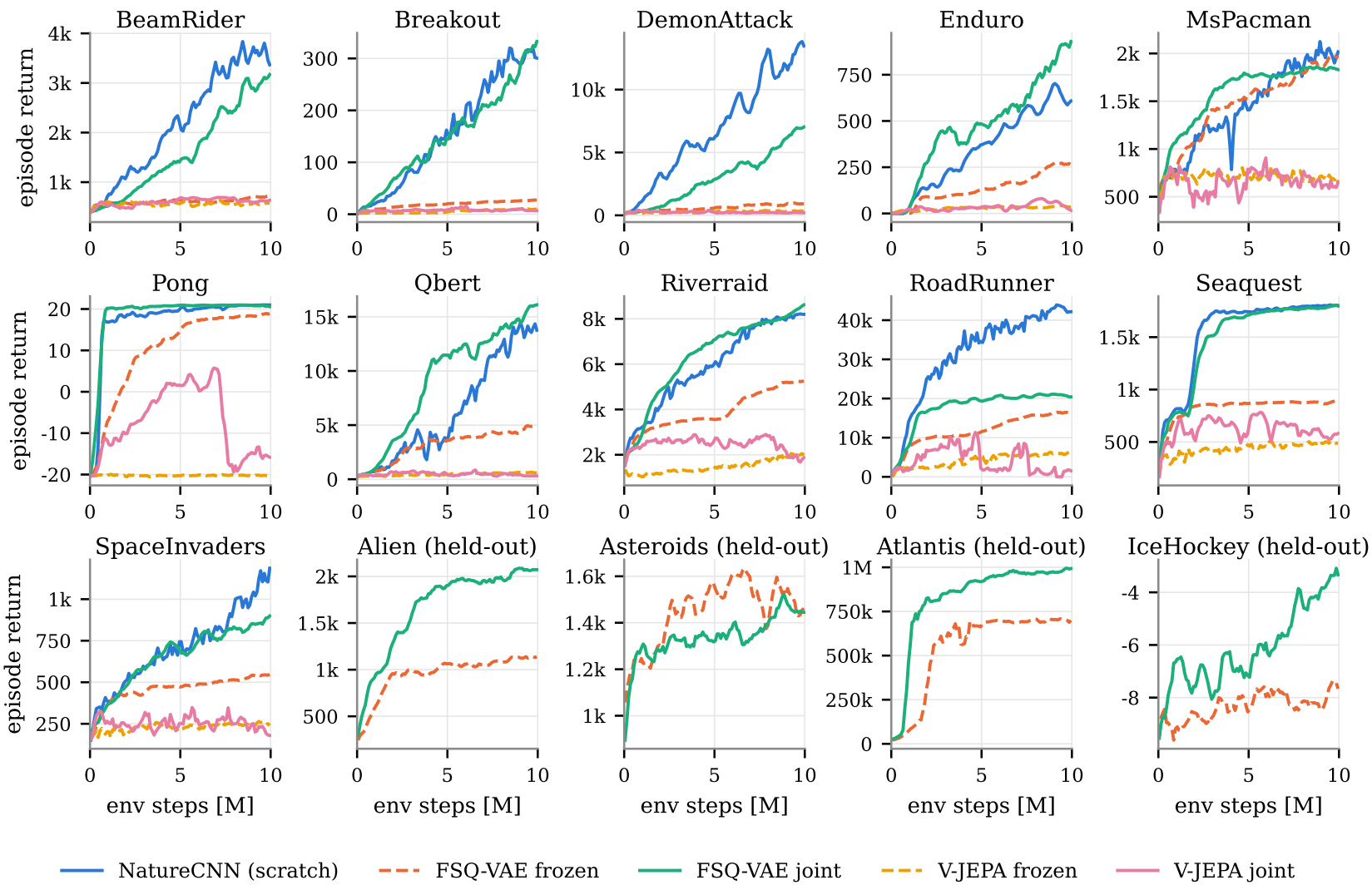
Results: Downstream RL

	NatureCNN	Std-VAE		Res-VQ		FSQ-VAE		V-JEPA	
	scratch	frzn	joint	frzn	joint	frzn	joint	frzn	joint
Med HNS (seen)	0.74	0.06	0.36	0.17	0.85	0.31	**1.07**	0.05	0.04
Mean HNS (seen)	**2.51**	0.14	0.53	0.30	0.91	0.54	2.02	0.13	0.07
Med HNS (held-out)	**0.50**	0.01	0.05	0.05	0.21	0.21	0.45	0.05	0.06

Std-VAE, Res-VQ and the NatureCNN/V-JEPA held-out scores were recorded rounded

- **FSQ fine-tuned:** highest median (1.07 vs. 0.74), beat NatureCNN on 4 of 11 games and nearly tied on Pong and Seaquest
- NatureCNN stayed ahead on the other games (about 2× on DemonAttack and RoadRunner), so its mean is higher
- **Other VAEs:** Std-VAE lowest on every game; Res-VQ below FSQ except on MsPacman, furthest behind on small, fast sprites
- **FSQ frozen:** learned something in every game, but reached less than a third of the fine-tuned median
- Fine-tuning beat freezing on 13 of 15 games (Breakout 12× higher)
- **Held-out games:** NatureCNN ahead of FSQ fine-tuned on 3 of 4
- **V-JEPA:** learned very little, frozen or fine-tuned

Results: Learning Curves



Solid: trained or fine-tuned encoders; dashed: frozen encoders

Results: Sample Efficiency

- I measured how many steps each agent needs to reach 50% and 80% of NatureCNN's final return
- At the 80% level, **FSQ fine-tuned was faster** on Enduro, MsPacman, Qbert and Riverraid
  - 1.4 to 2.6 M steps earlier (19–39% fewer steps)
- At the 50% level it was faster on 5 of 11 games
- NatureCNN was faster at both levels on the other six games; on MsPacman the FSQ agent learned faster but ended lower

Results: Can the V-JEPA Features Be Fixed?

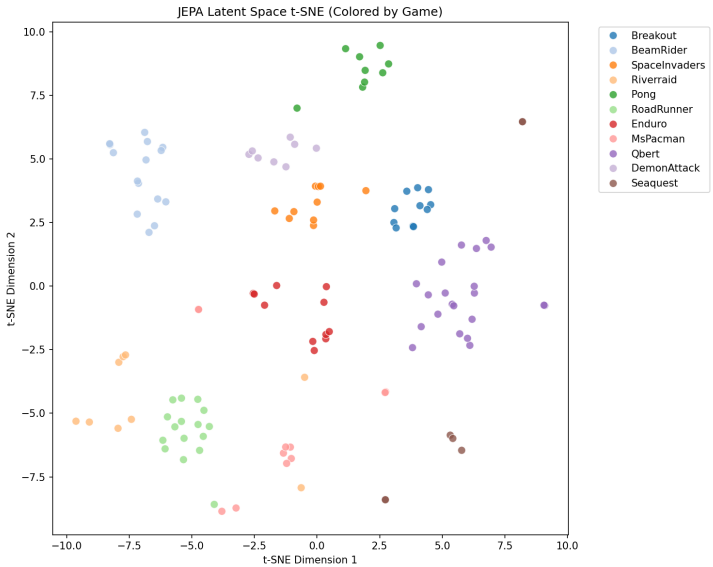
- My first guess: the 9 216-dimensional feature is too big for PPO
- So I repeated all 22 runs with the tokens averaged into a **256-dim** feature
- And the frozen runs once more with a 5× larger **entropy bonus**

Feature	Mode	Median HNS	Critic expl. var.
9 216 (flat)	frozen	0.045	0.73
256 (pooled)	frozen	0.021	0.53
256 (pooled)	fine-tuned	0.007	−0.27
256, entropy 0.05	frozen	0.015	0.03

- Neither change helped, so the size of the feature was not the problem; the information in it was

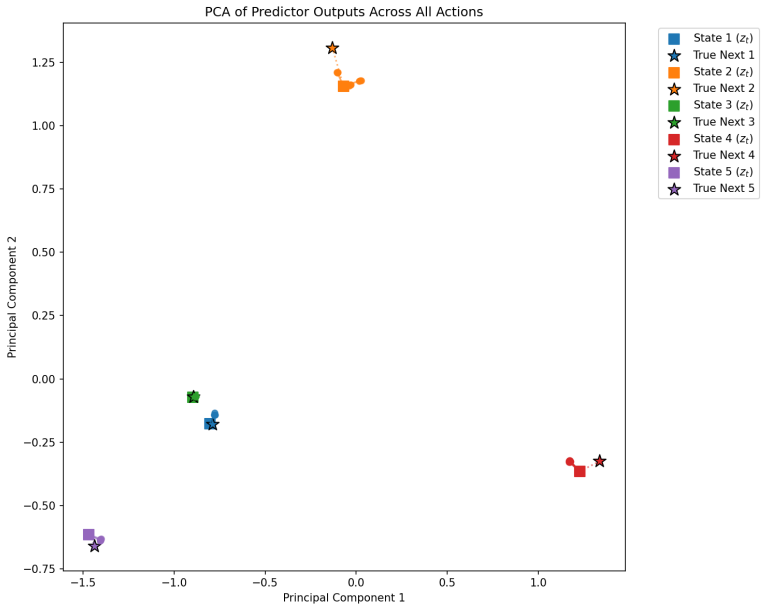
Results: World-Model Diagnostics

- Next states share 3 of 4 frames, so **copying** the current latent already looks good. I compared against that, an untrained model and **wrong actions**
- **No collapse:** effective rank 52.5 vs. 2.5 untrained
- **It learned dynamics:** better than copying, gap grows from +0.013 (1 step) to +0.046 (5 steps)
- **But the action barely matters:** a wrong action changes the prediction by only 0.006
- The game token gets about 2× uniform attention, but the encoder already recognises the game from the pixels
- On held-out games it is worse than copying



t-SNE of V-JEPA latents: they group by game

Results: Why the V-JEPA Failed with PPO



The predictions for all actions (dots) land almost on the same spot

What I measured

- Predictor mostly ignores the action
- Latents barely change per step (cos. 0.974)
- Latent space is sorted by game
- Worse on held-out games

What I think it means

- JEPA can skip details that are hard to predict, like a small ball
- 14×14 patches are coarse for small objects
- Expert data shows few other actions
- The critic could read how good a state is, but the policy could not find the right action

1 Introduction

2 Data & Games

3 Models

4 Methodology

5 Results

6 Conclusion

Conclusion

- **RQ1:** the FSQ-VAE gave the most useful features (beat Std-VAE on every game, Res-VQ on all but one)
- **RQ2:** fine-tuning worked much better than freezing
  - FSQ fine-tuned vs. NatureCNN: higher median HNS (1.07 vs. 0.74) and faster learning on 4 games, but a better final score on only 4 of 11
- **RQ3:** no pre-trained encoder helped on unseen games (NatureCNN ahead on 3 of 4)
- **RQ4:** freezing the V-JEPA made PPO about 25% faster (854 vs. 677 steps/s), but still slower than NatureCNN (976); the FSQ decoder shows what the features contain
- The V-JEPA learned how games usually evolve, but hardly what the agent's actions change
- **My answer:** a shared encoder is a useful starting point, but it cannot yet replace the NatureCNN when frozen

Limitations and Future Work

- **Limitations:** one seed per run, PPO not re-tuned for the pre-trained encoders
- **Future work:**
  - Repeat the main comparison with 3 seeds
  - V-JEPA with smaller patches and a loss where the action matters more (multi-step prediction, reward head)
  - Check the features first by predicting object positions, before running PPO
  - Use the FSQ grid inside a world model, and pre-train on more games

Thank You!